

Módulo 9 — Infraestructura y Despliegue de Sistemas de IA

Cloud vs on-premise, arquitecturas de despliegue, seguridad, latencia, vLLM, prompt caching y TCO.

BLOQUE 1 · INFRAESTRUCTURA DE IA EN PRODUCCIÓN

Módulo 9 — Infraestructura y Despliegue de Sistemas de IA

La infraestructura es el fundamento silencioso sobre el que operan los sistemas de IA. Un modelo que funciona perfectamente en un notebook de development puede fallar espectacularmente en producción si la infraestructura subyacente no está diseñada para manejar la variabilidad de los inputs reales, los picos de tráfico, los fallos de componentes externos, y los requisitos de latencia de los usuarios reales. La infraestructura de IA en 2025 es más compleja que la infraestructura de software tradicional porque incluye no solo los servicios de aplicación sino también los servicios de ML (modelos, APIs de LLM), las bases de datos vectoriales (para RAG), los orquestadores de herramientas (para agentes), y las plataformas de observabilidad específicas para LLMs.

Este módulo cubre las decisiones de infraestructura más importantes para los sistemas de IA: cloud vs on-premise, arquitecturas de despliegue, diseño de APIs, seguridad, escalabilidad, gestión de costes, y los patrones específicos para los sistemas de IA más comunes (RAG, agentes, fine-tuning y serving de modelos propios).

Cloud vs on-premise: la decisión fundamental

La decisión de dónde desplegar los sistemas de IA (cloud público, on-premise, o híbrido) tiene implicaciones en coste, latencia, privacidad, y capacidades disponibles. El cloud público (AWS, Azure, GCP) ofrece acceso inmediato a GPUs de última generación, escalado automático, y servicios gestionados para todos los componentes de la infraestructura de IA (AWS SageMaker, Azure ML, Google Vertex AI). Es la opción correcta para la mayoría de las organizaciones que empiezan con IA y no tienen razones especiales para on-premise. El on-premise es necesario cuando hay requisitos de soberanía de datos (los datos no pueden salir del datacenter propio), cuando el volumen de inferencia es tan alto que el coste de cloud supera al de infraestructura propia (el "cloud crossover point"), o cuando hay requisitos de latencia muy bajos que la red hacia el cloud no puede garantizar. La estrategia híbrida más común: usar APIs cloud para el desarrollo y los casos de uso de bajo volumen, y desplegar modelos propios on-premise para los casos de uso de alto volumen o alta sensibilidad de datos.

Arquitecturas de despliegue para sistemas de IA

Los patrones de arquitectura más comunes para sistemas de IA en producción son: el patrón API Gateway + LLM Backend (el más simple: una API REST que recibe las solicitudes del usuario, las procesa, llama al LLM, y devuelve la respuesta), el patrón de microservicios (cada componente del sistema de IA es un servicio independiente: servicio de RAG retrieval, servicio de LLM inference, servicio de herramientas del agente; permite escalar cada componente independientemente), el patrón event-driven (los eventos del negocio disparan workflows de IA de forma asíncrona: un nuevo contrato

subido dispara el análisis automático; más robusto ante fallos y picos de tráfico que el patrón síncrono), y el patrón serverless (funciones AWS Lambda o Azure Functions que se activan bajo demanda y escalan a cero cuando no hay tráfico; ideal para cargas intermitentes pero con límites en la duración de ejecución que pueden ser problemáticos para agentes de larga ejecución).

BLOQUE 2 · SEGURIDAD EN EL DESPLIEGUE

Seguridad de infraestructura para sistemas de IA

Gestión de secretos y credenciales

Los sistemas de IA en producción necesitan acceder a múltiples servicios externos (APIs de LLM, bases de datos, APIs de sistemas empresariales) usando credenciales que deben protegerse rigurosamente. Los anti-patrones más comunes y peligrosos son: almacenar las credenciales en el código fuente (hardcoded API keys en el código), en variables de entorno sin cifrar, o en archivos de configuración sin cifrar que se suben al repositorio de git. La práctica correcta es usar un gestor de secretos (HashiCorp Vault, AWS Secrets Manager, Azure Key Vault, o el sencillo pero efectivo Google Secret Manager) para almacenar las credenciales de forma cifrada, con rotación automática y acceso auditado.

El principio de mínimo privilegio aplica directamente a las credenciales de los sistemas de IA: cada servicio o componente debe tener solo las credenciales necesarias para su función específica. El agente de atención al cliente no debe tener acceso a las bases de datos financieras internas. El sistema de análisis de contratos no debe tener permisos de escritura en los sistemas de producción. Esta granularidad de permisos limita el daño potencial si un componente es comprometido o si sufre un ataque de prompt injection exitoso.

Rate limiting, autenticación, y protección contra abuso

Los sistemas de IA expuestos a usuarios externos (chatbots de cara al cliente, APIs de IA para terceros) deben protegerse contra abuso: usuarios que consumen desproporcionadamente los recursos (DoS económico por agotamiento de presupuesto de tokens), scrapers automatizados, y atacantes que intentan explotar el sistema. El rate limiting (limitar el número de solicitudes por usuario/IP en una ventana de tiempo) es la primera línea de defensa. La autenticación con JWT o API keys por usuario permite no solo control de acceso sino también auditoría del uso y revocación granular de acceso. El circuit breaker (desconectar automáticamente un componente que está fallando para evitar que sus fallos en cascada afecten al resto del sistema) es crítico para la resiliencia en sistemas de IA que dependen de servicios externos (APIs de LLM, bases de datos) que pueden tener downtimes o degradaciones.

BLOQUE 3 · ESCALABILIDAD, LATENCIA, Y COSTES

Optimizar el rendimiento y el coste en producción

Estrategias de optimización de latencia

La latencia en los sistemas de IA tiene múltiples fuentes que deben optimizarse de forma independiente. La latencia de primera respuesta (Time To First Token, TTFT) es crítica para la experiencia del usuario en sistemas conversacionales: el tiempo desde que el usuario envía la consulta hasta que recibe el primer token de la respuesta. El streaming de la respuesta (devolver los tokens al usuario según se generan, en lugar de esperar a que el modelo genere la respuesta completa) reduce

dramáticamente la TTFT percibida y hace los sistemas conversacionales mucho más responsivos. Las técnicas de aceleración de la inferencia incluyen: la cuantización del modelo (reducir la precisión de los parámetros de 16-bit a 8-bit o 4-bit, reduciendo el uso de memoria y aumentando la velocidad de inferencia con mínima pérdida de calidad), la selección de la GPU correcta (H100/A100 para máxima velocidad, L4/T4 para máximo valor coste-rendimiento), y los frameworks de serving optimizados (vLLM con PagedAttention para máximo throughput; TGI para alta disponibilidad).

El prompt caching es una técnica de reducción de coste y latencia disponible en las APIs de Anthropic, OpenAI, y Google: cuando el mismo prefijo de prompt (el system prompt más el contexto estático) se reutiliza en múltiples llamadas, el proveedor puede cachear los KV states de la atención para ese prefijo y evitar recalcularlos en cada llamada. Esto puede reducir el coste de las llamadas en un 80-90% para sistemas donde el system prompt es largo y constante (como los asistentes con RAG donde el contexto de los documentos es estático).

Gestión del coste total de propiedad (TCO)

El coste total de propiedad de un sistema LLM en producción incluye múltiples componentes que deben presupuestarse: coste de los tokens (input tokens × precio/token + output tokens × precio/token, que puede variar 100x entre modelos frontier y modelos pequeños), coste de infraestructura (servidores, GPU instances, bases de datos vectoriales, almacenamiento), coste de las herramientas de observabilidad y monitorización, y coste del equipo de ingeniería para mantenimiento y mejora continua. La gestión del TCO requiere: monitorización del gasto de tokens en tiempo real (alertas cuando el gasto diario supera el presupuesto), optimización del modelo por caso de uso (no todos los pasos del pipeline necesitan el modelo más caro), y evaluación periódica de si los modelos más nuevos y baratos pueden reemplazar a los más caros sin pérdida de calidad.

RESUMEN EJECUTIVO DEL MÓDULO 9: La infraestructura de IA incluye APIs de LLM, bases vectoriales, orquestadores de agentes, y plataformas de observabilidad. Cloud (AWS/Azure/GCP) es la opción default; on-premise cuando hay requisitos de soberanía de datos o volumen extremo. Los patrones de arquitectura son: API Gateway simple (lo más frecuente), microservicios (para escalar componentes independientemente), event-driven (para workflows asíncronos), y serverless (para cargas intermitentes). Seguridad: secretos en Vault/Secrets Manager, mínimo privilegio en credenciales, rate limiting + autenticación para APIs públicas, circuit breaker para resiliencia. Optimización de latencia: streaming (TTFT percibida), cuantización, vLLM/TGI para serving de alto throughput. Prompt caching reduce costes un 80-90% para system prompts largos y estáticos. TCO = tokens + infraestructura + observabilidad + equipo; monitorizar el gasto de tokens en tiempo real.